AIMS · RESEARCH DESIGN NOTE

Graph inductive bias in a Conditional Neural Process

A paper-faithful CNP baseline, a clean graph ablation, and a benchmark-held-out evaluation plan for online model testing.



Baseline first. Graph structure only after the pure CNP is verified.

Purpose

This note defines how to train a paper-faithful Conditional Neural Process (CNP) on the current filtered response data. It also specifies where a graph layer could be added without confusing the graph model with the CNP baseline.

Current implementation: actual training algorithm

This section describes what `codex_es_fixed_modal_cnp_train.py` actually does today. A **function realisation** is one filtered `(subject, benchmark)` task: its item embeddings are inputs and its right/wrong responses are outputs. It is not one response cell.

For the clean seed-20261018 split, there are 4,073 training tasks. One run has 3,000 outer iterations (optimizer updates), with a minibatch of 8 task episodes per iteration: 24,000 task exposures, or about 5.9 passes over the training tasks. The context/target split is newly sampled every time a task is exposed.

Loop level	Current value	Meaning
outer loop	3,000 steps	one AdamW update
inner loop	8 episodes per step	eight subject-functions in the minibatch
task scheduler	shuffled without replacement	every training task is used before the next shuffle
context size	uniformly 0 to 64	observed right/wrong labels available to the CNP
target size	at most 512	unobserved labels scored by the loss

For step `s = 1, …, 3,000` , the exact update is:

1 If the shuffled task queue is empty, replace it with a fresh permutation of all training tasks.

2 Pop 8 tasks from that queue. If the queue empties inside a batch, reshuffle immediately and continue; no task is skipped.

3 For each task `j` , randomly permute its eligible item-response pairs. Draw `k_j` uniformly from the integers 0 through `min(64, n_j − 1)` . The first `k_j` pairs are context `C_j` ; the next at most 512 pairs are target `T_j` .

4 Compute one Bernoulli target NLL for each task, then average the eight task losses.

5 Backpropagate the mean, clip the gradient norm to 5, and take one AdamW update.

$$\mathcal{L}_s = \frac{1}{8}\sum_{j=1}^{8} \mathrm{BCEWithLogits}\big(f_\theta(C_j, X_{T_j}), Y_{T_j}\big),$$

$$\theta \leftarrow \mathrm{AdamW}(\theta, \nabla_\theta \mathcal{L}_s).$$

The **current code uses disjoint contexts and targets**: `C_j ∩ T_j = ∅` . That matches the Active Evaluation paper's observed/unobserved split. It is not the original CNP reconstruction convention, where the context points are also included in the target set. The old `_episode` docstring says otherwise, but the actual slice `perm[k:k + 512]` is disjoint; the implementation, not that stale docstring, is the source of truth.

Every 50 steps, the fixed-validation copy evaluates the same bank of 64 held-out-subject episodes. It logs context reconstruction NLL, unseen-target NLL, and unseen-target Brier score; it restores the checkpoint with the lowest fixed-bank unseen-target Brier score. These are diagnostics and checkpoint selection, not extra optimizer updates.

The current bottleneck is `d = 64` : the hidden width and pooled representation `r` are 64-dimensional. The deterministic CNP run has no latent `z` ; the current optional latent-NP implementation defaults to `z ∈ ℝ³²` .

Paper-faithful reconstruction control

The baseline follows Conditional Neural Processes. One function is one subject on one benchmark. The model observes a small labelled context from that task and predicts the remaining responses.

Paper-faithful CNP

For task `k=(s,b)`, let `e_i` be the frozen embedding of item `i`, and let `y_i∈{0,1}` be the observed response. The model is

$$r_i = h_\phi([e_i, y_i]),$$

$$r_C = \frac{1}{|C|}\sum_{i \in C} r_i,$$

$$p(y_t = 1 \mid e_t, C) = \sigma\big(g_\psi([e_t, r_C])\big).$$

For an empty context, use `r_C=0`. The output is one Bernoulli logit per target item.

This baseline has:

- no IRT item parameters;
- no MAP estimate of theta;
- no latent variable;
- no attention or graph layer;
- no subject or benchmark embedding;
- no calibrator inside the model.

`CNPAIRT` is therefore not the paper-faithful baseline because it adds a 2PL item head and a MAP ability estimate. The closest existing implementation is `DeterministicCNP`, although the exact baseline should use independent encoder and decoder MLPs without LayerNorm or the optional GELU changes.

Data

Use the shared filtering and data rules under `organized/`:

1. Aggregate repeated subject-item observations to one binary response.
2. Remove duplicate item content.
3. Remove zero-variance training items.
4. Keep items answered by at least 30 distinct subjects.
5. Keep subject-benchmark tasks with at least 80 remaining items.
6. Leave missing responses absent. Never fill them with zero or a mean.

Use the frozen `me5_instruct-v2` embedding of `item_content` as `e_i`. The paper-faithful baseline uses no subject block. The context labels identify the task-specific function.

Do not create a permanent 128-item subset for a task. A training episode may contain at most 128 targets for memory control, but those targets must rotate across epochs so that every eligible item can contribute to training.

Duplicate `content_hash` values must not cross train, validation, and test folds. Otherwise an item can be text-identical to an item seen during training.

Architecture

Use the small architecture from the CNP paper as the starting point for this separate control:

- context encoder `h_φ`: three-layer ReLU MLP from 1,025 inputs to a

128-dimensional representation;

- aggregation: arithmetic mean;
- decoder `g_ψ`: three-layer ReLU MLP from `[e_t,r_C]` to one logit;
- optimizer: Adam;
- objective: Bernoulli negative log-likelihood.

This proposed control has `d = 128`, unlike the current training script's `d = 64`. GELU, LayerNorm, subject metadata, IRT anchors, and calibration are later ablations. They should not be included in the baseline called "paper-faithful CNP."

Episode construction for the paper-faithful control

Each task is one subject-benchmark response function. Sample tasks uniformly so that large response matrices do not dominate merely because they contain more cells.

For each epoch, visit every `(task, budget)` pair once, using budgets `0, 1, 3, 7, 15, 31` :

1. Choose a rotating target set `T` of at most 128 items from the task.
2. Choose a context `C⊆ T` at the requested budget.
3. Encode every labelled pair in `C`, then mean-pool the representations.
4. Predict the labels of every item in `T`, including the context items.
5. Average Bernoulli negative log-likelihood within the task.
6. Average the task losses within the optimizer batch.

The inclusion `C⊆ T` follows the original CNP objective. The model is asked to reconstruct both observed and unobserved values conditioned on the context. The target subset is a Monte Carlo estimate of the full-task likelihood, not a fixed training cap.

The existing v4.1 coverage scheduler is useful because it visits every task-budget pair and rotates item subsets. Its disjoint context-target rule is not the paper-faithful objective, so it should become a matched control rather than replace the main CNP arm.

Splitting and evaluation

The main evaluation must hold out complete benchmarks. Item-cell holdout only measures interpolation inside known benchmarks.

With seven filtered benchmarks, use benchmark-level cross-validation. Model weights, preprocessing choices, early stopping, and calibration must be fitted without the outer held-out benchmark.

At evaluation time:

1. Split each held-out task into a fixed acquisition pool and a disjoint

evaluation pool.

2. Use nested context prefixes at budgets `0, 1, 3, 7, 15, 31` .
3. Start with fixed random acquisition orders.
4. Report raw Brier score at every budget and the competition-weighted Brier

ALC.

5. Report proper probability ECE, AUC, and within-benchmark item pass-rate

correlation as diagnostics.

6. Fit any optional calibrator out of fold and report raw and calibrated

results separately.

The minimum comparison is:

Arm	Context-target relation	Extra structure
Beta-Binomial	disjoint evaluation	context rate only
Paper-faithful CNP	`C⊆ T` during training	none
Evaluation-matched CNP	`C∩ T=∅` during training	none

This comparison tells us whether the paper's reconstruction objective or the competition-matched disjoint objective transfers better. All other choices remain fixed.

Graph extension

The graph model is an ablation after the pure CNP works. It changes only the processing of context representations:

$$r_i = h_\phi([e_i, y_i]),$$

$$\tilde{r}_i = \mathrm{GraphLayer}(r_i, E_C),$$

$$r_C = \frac{1}{|C|} \sum_{i \in C} \tilde{r}_i.$$

The original decoder remains unchanged:

$$p(y_t = 1 \mid e_t, C) = \sigma\big(g_\psi([e_t, r_C])\big).$$

Use one residual graph-transformer layer. Construct edges only from information available at test time, such as cosine-nearest neighbours in the frozen item embedding space. Labels may enter node representations for acquired context items, but they must never determine the graph edges. Future target labels are never available.

If every context node attends to every other context node without an explicit or learned sparse relation, the model is self-attention rather than a GNN. Call it a graph model only when the edge construction carries an item-relation assumption.

The graph comparison should therefore be:

1. pure mean-pooled CNP;
2. CNP with one fully connected self-attention layer;
3. CNP with one embedding-neighbour graph layer.

Keep the decoder, episode stream, folds, random seeds, and target sets identical across the three arms. Any difference then belongs to the context relation operator rather than to another model or data change.

First low-cost check

Before any Modal run, train one fold and one seed locally on CPU. Confirm that:

- training and held-out-benchmark loss both improve;

- changing context labels changes target probabilities;
- target items receive gradients;
- all eligible items appear across rotating episodes;
- budget curves improve as context grows;
- the model learns non-trivial item ordering on the held-out benchmark.

Only after these checks should the experiment expand to all benchmark folds and three seeds.